🧠Swapping, Tuple, Dictionary & Complex

🧩 Meaning of “Swapping” in Programming

Swapping means **exchanging the values** of two variables. In other words — making variable `A` take the value of `B`, and variable `B` take the value of `A`.

💡 Example (Before and After Swapping)

```
a = 5
b = 10

# Before swapping:
# a = 5
# b = 10

# Swap using Python tuple unpacking
a, b = b, a

# After swapping:
# a = 10
# b = 5
```

🧠 What Happened Here?

The right side `(b, a)` creates a tuple of current values → `(10, 5)`. Python then assigns those values to `(a, b)` on the left side. So `a` gets `10` and `b` gets `5`.

🧮 Why Is It Useful?

- Sorting numbers or items (like in sorting algorithms).
- Temporarily switching values between two variables.

🔧 Another Example (Step-by-step with Temporary Variable)

```
a = 5
b = 10

# Swap using a temporary variable
temp = a
a = b
b = temp

print(a, b)  # Output: 10 5
```

> ✅ **In short:** Swapping means exchanging two values so that each variable takes the other’s value.

🧠 1️⃣ Tuple

💬 What is a Tuple?

A **tuple** is like a list, but it **cannot be changed** (we say it's *immutable*). That means — once you create it, you cannot add, remove, or change its items.

🧩 Example

```
# A tuple of student names
students = ("Alice", "Bob", "John")
print(students)

# Output:
# ('Alice', 'Bob', 'John')
```

⚙️ How to access tuple items

```
print(students[0])   # prints 'Alice'
print(students[-1])  # prints 'John'
```

🧰 Uses of Tuple

- When you want to store **fixed data** that should not change.
- Tuples use less memory than lists, so they are more efficient.
- Good for read-only data.

🌍 Real-Life Examples

```
# Days of the week
days = ("Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday")

# Coordinates on a map (x, y)
point = (10, 20)

# Storing constant information like (country code, country name)
country = ("RW", "Rwanda")
```

🧠 2️⃣ Dictionary

💬 What is a Dictionary?

A **dictionary** stores data in **key-value pairs**. Think of it like a real-life dictionary — you look up a *word* (key) to find its *meaning* (value).

🧩 Example

```
student = {
    "name": "Alice",
    "age": 20,
    "course": "Computer Science"
```

```
}

# Here:
# "name" is the key, and "Alice" is the value.
```

⚙️ Accessing values

```
print(student["name"])   # prints Alice

# Adding new data
student["grade"] = "A"

# Changing a value
student["age"] = 21
```

🧰 Uses of Dictionary

- When you need to store related data together.
- When you want to find information quickly using a **key**.
- Very useful in real-world applications like managing databases or APIs.

🌍 Real-Life Examples

```
# Student record
student = {"id": 1, "name": "Alice", "marks": 85}

# Contact list
contact = {"name": "John", "phone": "0789-123-456"}

# Product details
product = {"name": "Laptop", "price": 950.0, "in_stock": True}
```

🧠 3️⃣ Complex (Complex Numbers)

💬 What is a Complex Number?

A **complex number** has two parts: a **real part** and an **imaginary part.** In Python, you write it like this:

```
z = 3 + 4j
```

Here: `3` → Real part, `4j` → Imaginary part (notice the letter `j`, not `i` as in math)

⚙️ Accessing parts

```
print(z.real)  # Output: 3.0
print(z.imag)  # Output: 4.0
```

Math with complex numbers

```
a = 2 + 3j
b = 1 + 4j
print(a + b)  # Output: (3+7j)
```

🧰 Uses of Complex Numbers

- In engineering, physics, and mathematics for calculations involving waves, electricity, or signals.
- Used in scientific computing, such as electrical circuit analysis or control systems.

🌍 Real-Life Examples

- Electrical engineering: AC calculations use complex numbers to represent voltage and current.
- Robotics: complex numbers can represent rotation and movement.
- Data science: used in Fourier Transforms (audio and image processing).

✅ Summary Table

Data Type	Looks Like	Can Change?	Main Use	Real-Life Example
Tuple	("red", "blue", "green")	❌ No	Store fixed data	Days of the week
Dictionary	{"name": "Alice", "age": 20}	✅ Yes	Store key-value pairs	Student record
Complex	3 + 4j	✅ Yes	Math/Science calculations	Electrical signals